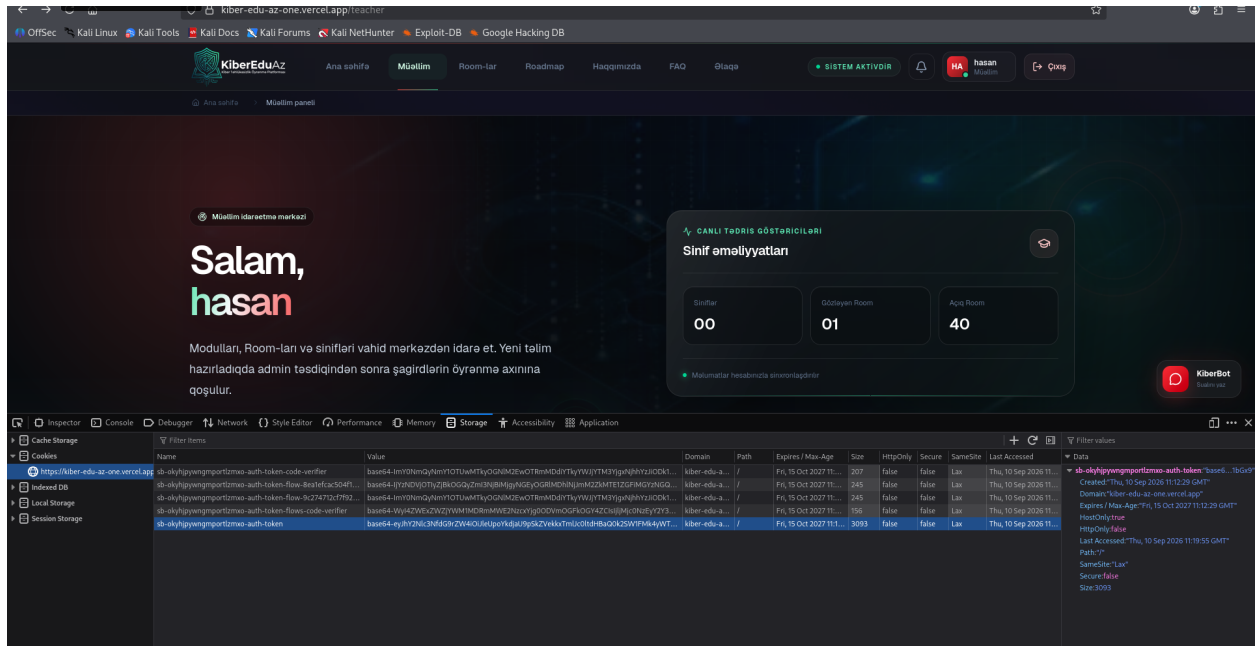


KiberEduAz platforması — [kiber-edu-az-one.vercel.app / teacher](https://kiber-edu-az-one.vercel.app/teacher)

Ümumi risk səviyyəsi: YÜKSƏK — istifadəçi sessiyalarının oğurlanması mümkündür.

Şəkil 1 — sb-...-auth-token-code-verifier cookie-si: HttpOnly=false, Secure=false



Şəkil 2 — bütün Supabase auth cookie-ləri (auth-token, auth-token-flow-*, auth-token-flows-code-verifier) eyni problemlı ayarlarla: HttpOnly=false, Secure=false, SameSite=Lax

2. Aşkar edilmiş problemlər

2.1. HttpOnly = false (auth cookie-lərində)

Risk səviyyəsi: KRİTİK

Bütün auth cookie-ləri (sb-okyhjpywngmportlzmxo-auth-token və törəmələri) HttpOnly bayrağı olmadan qoyulub. Bu o deməkdir ki, səhifədə işləyən istənilən JavaScript kodu document.cookie vasitəsilə bu token-lərə birbaşa çata bilər. Əgər saytda hər hansı bir XSS (Cross-Site Scripting) boşluğu — məsələn istifadəçi girişli sahədə (ad, şərh, mesaj sahəsi) təmizlənməmiş HTML/JS icra oluna bilirsə — hücumçu bir sətirlik skriptlə bütün müəllim/admin sessiyasını oğurlaya, sonra öz brauzerinə köçürüb həmin istifadəçi kimi sistemə daxil ola bilər (session hijacking).

2.2. Secure = false

Risk səviyyəsi: YÜKSƏK

Cookie-lərdə Secure bayrağı işarələnməyib. Nəzəri olaraq bu, cookie-nin yalnız HTTPS deyil, HTTP (şifrələnməmiş) bağlantı üzərindən də göndərilə biləcəyi mənasına gəlir. Açıq Wi-Fi şəbəkəsində və ya hər hansı downgrade / man-in-the-middle ssenarisində trafik izlənersə, token asanlıqla ələ keçirilə bilər. Sayt hazırda HTTPS altında işləsə belə, bu bayraq olmadığı üçün brauzer əlavə mühafizə tətbiq etmir.

2.3. Token-lərin kodlanma forması (Base64 JWT)

Risk səviyyəsi: ORTA

Cookie dəyərləri base64-... prefiksi ilə saxlanılan Supabase sessiya obyektləridir (içərisində JWT access/refresh token, istifadəçi ID-si, rol və s.). Base64 şifrələmə deyil, sadəcə kodlamadır — kim ki bu dəyərə çatır, heç bir açar olmadan onu decode edib içindəki məlumatları (istifadəçi ID, rol, son istifadə tarixi) oxuya bilər. Bu özlüyündə boşluq deyil (Supabase-in standart formatıdır), lakin yuxarıdakı HttpOnly/Secure problemləri ilə birləşəndə, oğurlanmış token dərhal istifadəyə hazır olur.

2.4. "code-verifier" cookie-sinin uzun ömürlü olması

Risk səviyyəsi: ORTA

PKCE OAuth axınına aid auth-token-code-verifier və auth-token-flows-code-verifier cookie-ləri normal halda müvəqqəti olmalı və login prosesi tamamlanan kimi silinməlidir. Lakin bu cookie-lərin Expires dəyəri 2027-ci ilə qədər (~1 il) təyin olunub. Uzun ömürlü qalması hücum səthini artırır — bu cookie-lər uzun müddət brauzerdə/köşkdə (kiosk) qalaraq başqa şəxs tərəfindən əldə oluna bilər.

2.5. SameSite=Lax

SameSite=Lax OK səviyyədədir (Strict qədər sərt deyil, amma CSRF-in bir çox formasına qarşı qismən qoruma verir). Bu problem deyil, sadəcə qeyd üçündür — əgər cross-site sorğu tələb olunmursa, Strict-ə keçirmək daha təhlükəsizdir.

3. Developerlər üçün düzəliş planı (fix)

Aşağıdakı addımlar prioritet sırası ilə verilib. 1 və 2-ci addımlar dərhal (kritik) tətbiq olunmalıdır.

3.1. Cookie-ləri HttpOnly + Secure edin

Supabase Auth ilə işləyərkən sessiya cookie-lərinin server tərəfində (backend/middleware) idarə olunması tövsiyə olunur ki, brauzer JS-i onlara çata bilməsin. Next.js/Vercel mühitində @supabase/ssr paketindən istifadə edərək cookie-ləri server tərəfdə belə qurun:

```
cookieOptions: {  
  name: 'sb-auth-token',  
  httpOnly: true,  
  secure: true,  
  sameSite: 'strict',  
  maxAge: 60 * 60 * 8 // 8 saat, uzun müddət yox  
}
```

Qeyd: client-side createBrowserClient istifadə etdikdə cookie-lər defolt olaraq JS-dən oxuna bilən formada qalır. Buna görə auth axınını createServerClient (middleware.ts / route handler) üzərindən keçirmək və access token-i client-ə birbaşa ötürməmək lazımdır.

3.2. Bütün mühitdə HTTPS + HSTS məcburi edin

Vercel-də domain artıq HTTPS altındadır, amma Secure bayrağını cookie səviyyəsində aktiv etməklə yanaşı, tətbiq başlığına (headers) aşağıdakını əlavə edin ki, brauzer heç vaxt HTTP-yə keçməsin:

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

3.3. XSS-ə qarşı qoruma (Content Security Policy)

HttpOnly düzəldilsə belə, XSS-in özünü bağlamaq vacibdir (fişinq, DOM manipulyasiyası, digər token-ə bənzər məlumatların oğurlanması riski qalır). next.config.js-də CSP başlığı əlavə edin və bütün istifadəçi girişli sahələri (ad, room adı, şərhlər) server tərəfdə sanitize edin (məs. DOMPurify):

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none';
```

3.4. Token ömrünü qısaldın və refresh məntiqini nəzərdən keçirin

Access token üçün qısa ömür (15-60 dəqiqə), refresh token üçün isə HttpOnly+Secure cookie-də saxlanan, rotasiya olunan (hər istifadədən sonra yenilənən) strategiya tətbiq edin. "code-verifier" cookie-ləri isə OAuth axını bitən kimi (uğurlu və ya uğursuz nəticədən asılı olmayaraq) dərhal silinməli və heç vaxt 1 illik Expires almamalıdır — bunun üçün Supabase client konfiqurasiyasında flowType: 'pkce' ilə birgə cookie maxAge-i qısa (məs. 10 dəqiqə) təyin edin.

3.5. Server tərəfdə sessiya yoxlaması

Client-də saxlanan token-ə etibar etməkdənsə, hər həssas sorğuda (məs. "Room yarat", "Şagird təsdiqlə") server tərəfdə supabase.auth.getUser() ilə token-i yenidən doğrulayın və rol yoxlaması (RLS — Row Level Security) Supabase tərəfində aktiv olduğuna əmin olun. Beləliklə, token oğurlansa belə, məlumat bazası səviyyəsində əlavə maneə olur.

3.6. Monitoring və alert

Anormal giriş fəaliyyətini (eyni token ilə fərqli IP/coğrafiyadan istifadə, qısa müddətdə çoxlu uğursuz login cəhdi) izləmək üçün Supabase Auth logs və ya Vercel/Sentry integrasiyası ilə alert qurun.

4. Xülasə cədvəli

Problem	Risk	Təvsiyə edilən fix
HttpOnly = false	Kritik	Cookie-ləri server tərəfdə HttpOnly təyin edin
Secure = false	Yüksək	Secure=true + HSTS başlığı
Uzun ömürlü code-verifier	Orta	Qısa maxAge, OAuth bitəndə sil
Potensial XSS səthi	Yüksək	CSP başlığı + input sanitizasiyası (DOMPurify)
SameSite=Lax	Aşağı	Mümkünsə Strict-ə keçirin

Qeyd: Bu hesabat müşahidə edilən brauzer konfigurasiyasına əsaslanır və məsuliyyətli açıqlama (responsible disclosure) məqsədi daşıyır. İstismarı (exploit) təsvir etmir, yalnız aşkar olunmuş konfigurasiya problemlərini və düzəliş yollarını göstərir.

Pentest Hesabatı

KiberEduAz — Broken Access Control / Privilege Escalation

Hədəf (Target)	kiber-edu-az-one.vercel.app (frontend) / kiberedu-api.onrender.com (backend)
Test tarixi	10 Sentyabr 2026
Test növü	Grey-box kod review + canlı sistemdə doğrulama (authenticated PoC)
Test edən	Tətbiqin sahibi (öz-app pentest təcrübəsi məqsədilə)
Ümumi risk səviyyəsi	YÜKSƏK (High)
Tapılan boşluq sayı	1 əsas (2 endpoint qrupunda) + hardening tövsiyələri

Bu hesabat KiberEduAz tədris platformasının backend kodunun (NestJS + Prisma + Supabase) təhlükəsizlik baxımından nəzərdən keçirilməsi və tapılan boşluğun canlı (production) mühitdə, real hesabla, təhlükəsiz şəkildə doğrulanması əsasında hazırlanmışdır.

1. İcmal (Executive Summary)

Kod review zamanı backend-in rooms (otaqlar) modulunda tətbiq olunmuş sahiblik və rol yoxlamalarının, eyni məntiqi tələb edən paths (kurikulum yolları) və modules (modullar) endpoint-lərinə tətbiq edilmədiyi aşkarlanmışdır. Bunun nəticəsində, admin tərəfindən təsdiqlənmiş istənilən TEACHER (müəllim) rollu hesab: admin təsdiqi olmadan istənilən DRAFT (qaralama) path/modulu birbaşa PUBLISHED (dərc edilmiş) vəziyyətinə gətirə bilər — bu, tələbələrə nəzərdən keçirilməmiş məzmunun göstərilməsinə səbəb olur. Həmçinin özünə aid olmayan (başqa müəllimin yaratdığı) path və ya modulu redaktə edə bilər.

Boşluq 10 Sentyabr 2026 tarixində canlı backend-də (kiberedu-api.onrender.com) real, admin-təsdiqli müəllim hesabı ilə sınaq edilmiş və tam təsdiqlənmişdir: seçilmiş bir DRAFT modul uğurla PUBLISHED edilmiş, sonra sistemi orijinal vəziyyətinə qaytarmaq üçün dərhal geri DRAFT edilmişdir. Test zamanı heç bir real tələbə məlumatına təsir edilməmişdir.

#	Boşluq	Ciddiyyət	Status
1	Path/Module publish workflow bypass (privilege escalation)	Yüksək	Canlıda təsdiqləndi
2	Path/Module-da sahiblik (ownership) yoxlamasının olmaması	Orta	Kod review ilə aşkarlandı
3	Rooms modulunda tapşırıq (task) səviyyəsində sahiblik yoxlamasının olmaması	Orta	Kod review ilə aşkarlandı

2. Əhatə Dairəsi və Metodologiya

2.1 Əhatə dairəsi

Frontend: Next.js 15 (App Router), Supabase Auth (SSR client), Vercel-də deploy olunub. Backend: NestJS + Prisma ORM, Render-də deploy olunub (kiberedu-api.onrender.com). Verilənlər bazası: PostgreSQL (Supabase). Autentifikasiya: Supabase Auth (JWT, JWKS ilə asimmetrik doğrulama).

2.2 Metodologiya

Statik kod review (SAST-tipli, əl ilə): autentifikasiya, rol/icaazə yoxlaması, mass assignment, IDOR, injection, XSS, CORS, sessiya idarəetməsi kateqoriyaları üzrə bütün backend controller/service faylları və frontend auth/API integrasiya qatı nəzərdən keçirildi. Dinamik doğrulama (canlı mühitdə): aşkarlanan hipotez real, admin-təsdiqli TEACHER hesabı ilə Supabase Auth REST API-dən alınan JWT access token istifadə edilərək backend-ə birbaşa HTTP sorğuları (curl) ilə sınıandı. Təsirin minimuma endirilməsi: yalnız boş (0 tapşırıqlı) bir DRAFT modul hədəf seçildi, dəyişiklik dərhal geri qaytarıldı, heç bir tələbə/istifadəçi məlumatı oxunmadı, dəyişdirilmədi və ya silinmədi.

3. Tapıntı #1 — Publish Workflow Bypass (Privilege Escalation)

YÜKSƏK (High) · CVSS 3.1 Təxmini: 7.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N)

CWE: CWE-863 (Incorrect Authorization), CWE-269 (Improper Privilege Management) · OWASP: A01:2021 — Broken Access Control

3.1 Təsvir

Backend-in rooms modulunda dərc etmə (publish) əməliyyatı şüurlu şəkildə yalnız ADMIN roluna açıqdır və status sahəsi qeyri-admin istifadəçilər üçün sorğudan silinir (bax: rooms.service.ts, orijinal şərh: “Teachers always create drafts; only an admin can publish later”). Amma eyni qayda paths.service.ts faylındakı updatePath() və updateModule() funksiyalarına tətbiq edilməyib — bu funksiyalar dto.status dəyərini heç bir rol yoxlaması olmadan birbaşa verilənlər bazasına yazır.

3.2 Zəif Kod (əvvəl)

```
// backend/src/catalog/paths.service.ts
async updatePath(id: string, dto: Partial<UpsertPathDto>) {
  return this.prisma.path.update({
    where: { id },
    data: this.pathData(dto), // dto.status filtrsiz keçir!
  });
}

private pathData(dto: Partial<UpsertPathDto>) {
  return { ..., status: dto.status, ... }; // rol yoxlaması yoxdur
}
```

3.3 Təsdiqlənmiş Exploit (Proof of Concept)

Canlı sistemdə, admin-təsdiqli TEACHER hesabı ilə (parol/token gizlədilib):

```
# 1) Supabase-dən JWT access_token alındı (Auth REST API)
POST https://<supabase-ref>.supabase.co/auth/v1/token?grant_type=password

# 2) Profil təsdiqləndi:
GET /api/v1/profiles/me
-> { "role": "TEACHER", "accountStatus": "ACTIVE", ... }

# 3) Mövcud, DRAFT statuslu bir modul tapıldı (id: e9110cb5-...),
# bu modulun admin tərəfindən təsdiqi/yoxlanışı YOXDUR.

# 4) EKSPLOYT: admin təsdiqi olmadan modulu PUBLISHED etdik:
PATCH /api/v1/modules/e9110cb5-cbfa-41bd-a46b-95876fbccbd
Authorization: Bearer <TEACHER_TOKEN>
Content-Type: application/json
{"pathId":"41704815-...","slug":"dd","title":"dd","status":"PUBLISHED"}
-> HTTP 200 OK -- status dəyişdi: DRAFT -> PUBLISHED

# 5) TƏSDİQ: GET /api/v1/paths sorğusu ilə modulun həqiqətən
# PUBLISHED olduğu görüldü.

# 6) TƏMİZLİK: eyni PATCH sorğusu status:"DRAFT" ilə təkrar
# göndərilərək sistem orijinal vəziyyətinə qaytarıldı.
```

Nəticə: Exploit birbaşa canlı production backend-də uğurla icra edildi və doğrulandı.

3.4 Təsir (Impact)

Redaksiyadan keçməmiş, natamam və ya səhv məzmun tələbələrə birbaşa göstərilə bilər. Admin-in məzmun keyfiyyətinə nəzarət mexanizmi tamamilə bypass olunur. Hər hansı bir müəllim hesabının kompromis olunması halında, təcavüzkar bütün kurikulumu idarə edə bilər. Eyni zəiflik sinfindən (bax Tapıntı #2) istifadə edərək, başqa müəllimlərin məzmunu da dəyişdirilə/silinə bilər.

3.5 Düzəliş (Remediation)

Rooms modulundakı qoruma məntiqi paths/modules-a da tətbiq edilməlidir: (1) status sahəsi yalnız ADMIN rolu üçün icazəli olmalı, qeyri-adminlər üçün undefined edilməlidir; (2) yeni yaradılan məzmun rol nə olursa olsun defolt olaraq DRAFT olmalıdır.

```
// Düzəldilmiş: backend/src/catalog/paths.service.ts
async updatePath(user: AuthenticatedUser, id: string, dto: Partial<UpsertPathDto>) {
  await this.assertCanManagePath(user, id);
  // Qeyri-adminlər status-u heç vaxt bu endpoint vasitəsilə dəyişə bilməz.
  const safeDto = user.profile.role === UserRole.ADMIN
    ? dto
    : { ...dto, status: undefined };
  return this.prisma.path.update({ where: { id }, data: this.pathData(safeDto) });
}
```

Qeyd: Tam düzəldilmiş fayllar (paths.service.ts, paths.controller.ts, rooms.service.ts, rooms.controller.ts) ayrıca təqdim edilib və artıq hazırdır — Əlavə A-ya bax.

4. Tapıntı #2 — Sahiblik (Ownership) Yoxlamasının Olmaması

ORTA (Medium) · CVSS 3.1 Təxmini: 5.4 (AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N)

CWE: CWE-639 (Authorization Bypass Through User-Controlled Key) · OWASP: A01:2021 — Broken Access Control

4.1 Təsvir

PathsService.updatePath() və updateModule() funksiyaları heç bir yerdə createdById (yaradıcı) sahəsini yoxlamır. Müqayisə üçün, ClassesService.assertCanManage() funksiyasında bu yoxlama düzgün tətbiq olunub — o cümlədən “Bu sinif sənə aid deyil” xəta mesajı ilə. Paths/Modules-da isə istənilən aktiv TEACHER, ID-ni bilməklə (məsələn GET /api/v1/paths vasitəsilə), başqa müəllimin yaratdığı məzmunu redaktə edə bilər.

4.2 Düzəliş

Hər update əməliyyatından əvvəl sahibliyi yoxlayan bir mühafizəçi metod əlavə edildi:

```
private async assertCanManagePath(user: AuthenticatedUser, pathId: string): Promise<void> {
  if (user.profile.role === UserRole.ADMIN) return;
  const path = await this.prisma.path.findUnique({
    where: { id: pathId }, select: { createdById: true },
  });
  if (!path) throw new NotFoundException('Path tapılmadı');
  if (path.createdById !== user.id) {
    throw new ForbiddenException('Bu path sənə aid deyil');
  }
}
```

5. Tapıntı #3 — Rooms Modulunda Tapşırıq Səviyyəsində Sahiblik

ORTA (Medium)

CWE: CWE-639, CWE-284 (Improper Access Control)

RoomsService.update(), upsertTask() və removeTask() funksiyalarında da eyni sahiblik yoxlaması yox idi. Əlavə olaraq, upsertTask(roomId, dto, taskId) funksiyası verilən taskId-nin doğrudan roomId-ə aid olduğunu yoxlamırdı — nəzəri olaraq, A room-u idarə edən müəllim, URL-də A room ID-sini göstərərək B room-a aid bir task-ı redaktə/silə bilərdi.

Düzəliş: assertCanManageRoom() mühafizəçisi əlavə edildi və hər task əməliyyatında task.roomId === roomId yoxlaması tətbiq edildi.

6. Əlavə Hardening Təvsiyələri

Aşağıdakılar kritik zəiflik deyil, lakin ümumi təhlükəsizlik səviyyəsini artıracaq təvsiyələrdir:

Təvsiyə	Səbəb
Login/register route-larına ayrıca, sərt rate-limit (@Throttle) qoyulması	Hazırkı 120 req/dəq qlobal limit brute-force həmləsi üçün kifayət qədər sərt deyil.
Admin əməliyyatlarına (rol dəyişmə, publish/unpublish) audit log əlavə edilməsi	Kim, nə vaxt, nə dəyişdi sualına cavab vermək və insident zamanı iz sürmək üçün.
Supabase-də RLS (Row Level Security) policy-lərinin yoxlanılması	Backend Prisma ilə service-role açarından istifadə edir, amma client-side birbaşa çağırışlar üçün RLS mühüm əlavə qoruma qatıdır.
Helmet-in Content-Security-Policy sazlamasının açıq şəkildə konfigurasiya edilməsi	Defolt helmet() bəzi hallarda CSP-ni deaktiv saxlayır.

7. Aşkarlanan Güclü Tərəflər (Positive Findings)

Ədalətli olmaq üçün qeyd edilməlidir ki, kod bazasının böyük hissəsi diqqətlə və düzgün yazılıb:

- JWT JWKS ilə doğrulanır, alg:none bypass mümkün deyil; rol həmişə JWT-dən deyil, DB-dən oxunur.
- UpdateProfileDto-da rol sahəsi yoxdur — mass assignment ilə özünü admin edə bilməzsən.
- ValidationPipe global whitelist:true, forbidNonWhitelisted:true ilə işləyir.
- Sinif (classes) və bildiriş (notifications) modullarında IDOR-un qarşısı düzgün alınıb.
- Supabase middleware getUser() istifadə edir (getSession() yox) — cookie spoofing-in qarşısı alınıb.
- Heç bir yerdə raw SQL, eval(), command injection nümunəsi tapılmadı — Prisma ORM düzgün istifadə olunub.
- Sual cavablarının doğru/yanlış qərarı yalnız server tərəfdə verilir; düzgün cavab tələbə həll edənə qədər client-ə göndərilmir.

8. Nəticə və Növbəti Addımlar

Təcili (bugün): Əlavə A-dakı 4 fayl backend-ə tətbiq edilməli və Render-ə deploy edilməlidir.

Deploy sonrası: 3.3-dəki PoC yenidən icra edilməli — nəticə 403 Forbidden olmalıdır.

Təhlükəsizlik gigiyenası: Test zamanı istifadə olunan hesabın parolu dəyişdirilməli, Supabase Dashboard-dan aktiv sessiyalar ləğv edilməlidir.

Orta müddətli: Bölmə 6-dakı hardening tövsiyələri təqvimə salınmalıdır.

Bu hesabat, tətbiqin sahibi tərəfindən öz sistemində, öz razılığı ilə aparılan qiymətləndirmə əsasında hazırlanmışdır. Sənəddə istifadə olunan token və parol kimi məlumatlar gizlədilmişdir.

Qeyd: Bu sənəd əvvəlki versiyada şrift/kodlaşdırma problemi (Azərbaycan hərflərinin görünməməsi) səbəbindən oxunaqsız olan orijinal pentest hesabatının məzmunca eyni, düzgün kodlaşdırılmış və səliqəyə salınmış versiyasıdır.